Lab 7: Fuzzing for Input Validation Bugs

H K Manaswin | PES1UG24AM108 | Program: L6_L7_42 Salon Booking

Repository: https://github.com/SETAPESU26/L6_L7_42_salon_booking

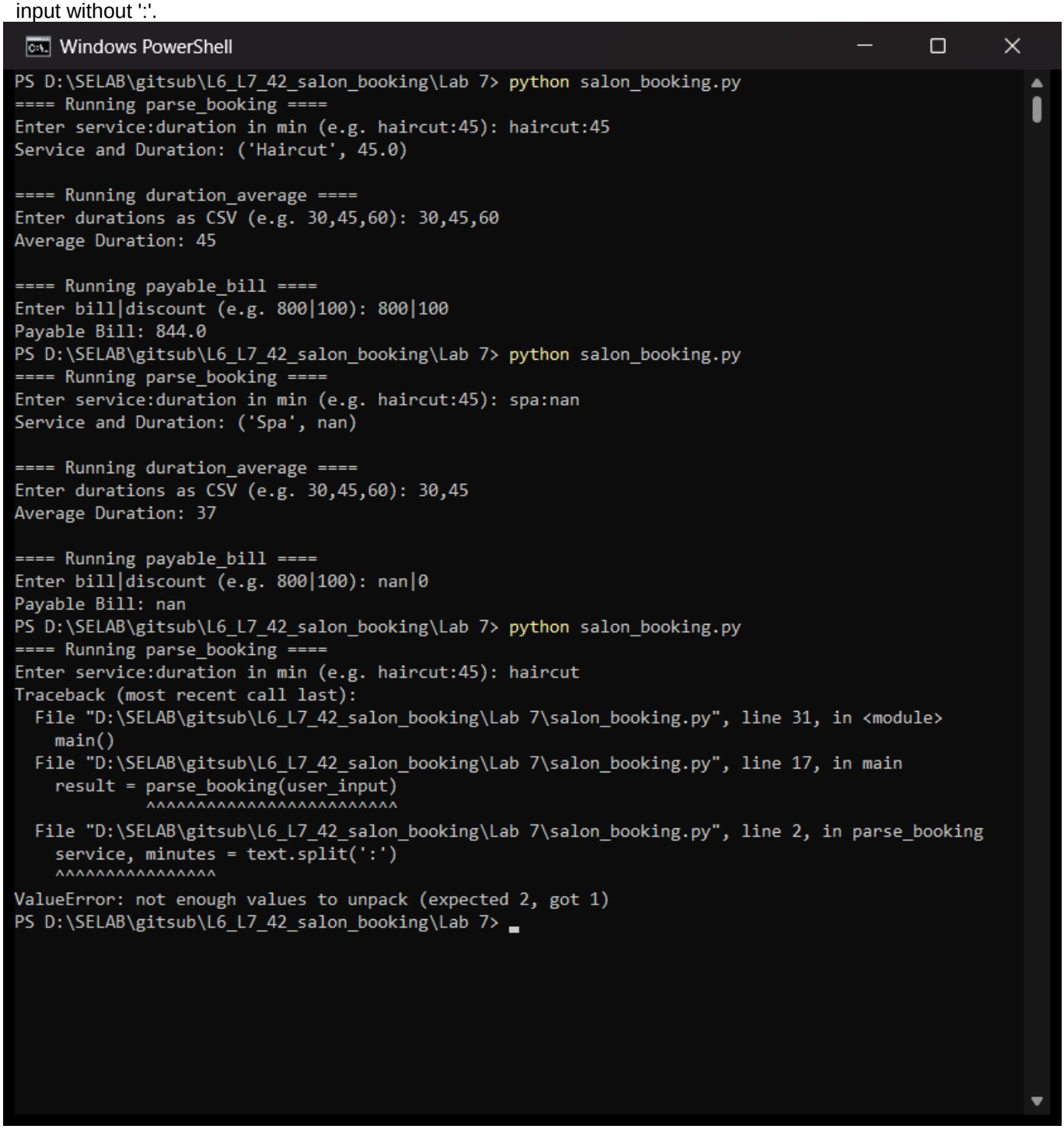
Bugs found by Hypothesis and how they were fixed

Function	Falsifying input (example)	Bug	Fix
parse_booking	None, '', ':', 'a:b:c'	AttributeError, unpack ValueError, float('') crash	Type check; exactly one ':'; numeric part parsed safely; return None
parse_booking	'spa:nan', 'a:-30', ':45'	nan, negative and zero minutes, empty service accepted	Require non-empty service and finite 0 < minutes <= 1440
duration_average	None, '', '1,,3', 'abc'	int('') / int('abc') crash, AttributeError on None	Every item must be a decimal integer, else return None
duration_average	'1,2' gives 1; '0'	Floor division (//) truncates the average; zero/negative durations accepted	True mean (/) rounded to 2 places; each value 1..1440
payable_bill	None, '', '1|2|3', 'a|b'	Unpack and float conversion crashes	Type check; exactly one '|'; safe float parsing; return None
payable_bill	'NAN|0', 'inf|inf', '1e308|0'	nan / inf escape through max(total, 0); overflow	Reject non-finite parts and non-finite totals
payable_bill	'0.00|-0.01'	Negative discount (or bill) accepted, increasing the bill	Reject negative bill or discount

Test design (test_salon_booking.py): for each function a no-crash test on `st.text() | st.none()` plus text biased towards ':', '|', ',', signs, digits and 'nan'/'inf'; an output-validity test (result is None, or finite and in range); and correctness tests on valid generated input (round-trip parsing, true mean between min and max, bill = max(bill x 1.18 - discount, 0), negative discount rejected).

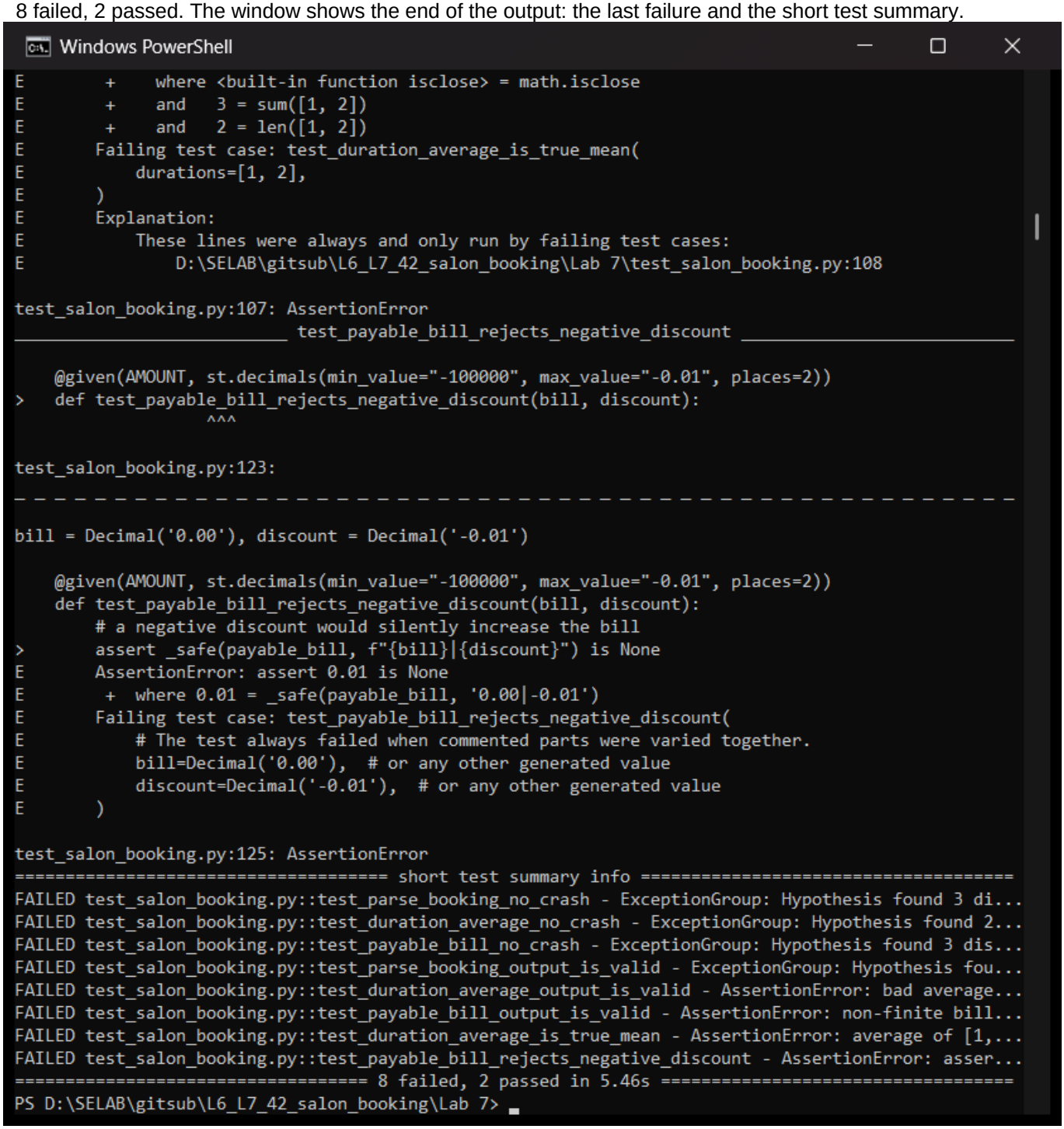
Step 2: Running the buggy application (python salon_booking.py)

Run 1: valid input works. Run 2: silent bugs (nan accepted, average 37 instead of 37.5, bill nan). Run 3: crash on input without ':'.



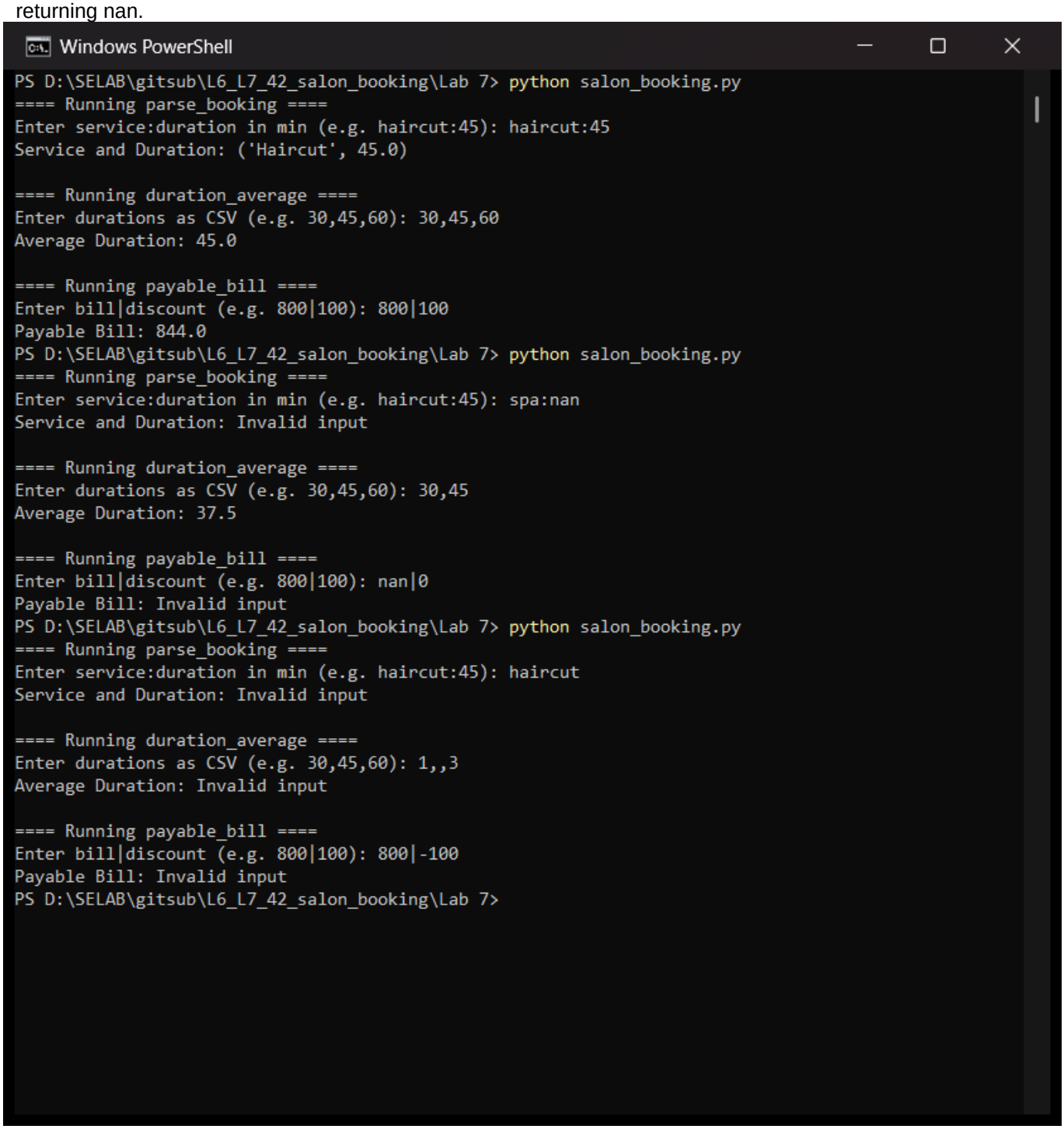
Step 4: Hypothesis tests failing on the buggy application (pytest test_salon_booking.py)

8 failed, 2 passed. The window shows the end of the output: the last failure and the short test summary.



Step 5: Running the fixed application (python salon_booking.py)

Valid input gives the expected results (average now 37.5); invalid input prints 'Invalid input' instead of crashing or returning nan.



Step 6: Hypothesis tests on the fixed application (pytest test_salon_booking.py)

All 10 tests pass (300 generated examples per test).

